



**Frédérique Michèle Rey**

Humboldt Universität zu Berlin

# Introduction to Python

Seance 2

March 16, 2026

- 1 Why learn to program?
- 2 What is Python ?
- 3 Why Python?
- 4 Example
- 5 Installing Python
- 6 Working with Jupyter
- 7 Practicalities for labs
- 8 Overview of the two sessions

- PhD & Habilitation in Biblical Studies (Strasbourg and KULeuven)
- Actually Prof. at the Humboldt University of Berlin (chair of Biblical Theology)
- **Hebrew and Greek Philology** : Critical Edition of the Hebrew Manuscripts of Ben Sira and of the Dead Sea Scrolls
- **Computational Philology** : PI of the ANR Project Sherbet (Stemmatology of the Hebrew Bible Transmission)
- frederique.rey@hu-berlin.de

# Why learn to program?

---

## The challenge of digital humanities

- The advancement of computing power
- The extremely rapid impact and development of artificial intelligence in our daily lives

## An imperative

It is becoming unthinkable not to integrate these new technologies when working on corpora and metadata too vast to be analyzed manually.

## With Python, you can:

- Analyze **thousands of texts and metadata** in seconds
- Automate repetitive tasks
- Produce **reproducible** analyses
- Visualize linguistic data

## Examples in philology

- Encoding editions
- Automatically performing collations
- Applying phylogenetic algorithms to reconstruct stemmata
- Conducting fine-grained stylometric analyses to identify authors
- Automatically detecting quotations or allusions
- Performing quantitative analyses on vocabulary or semantics

# What is Python ?

---

## Programming languages

The goal of a **programming language** is to write instructions in a way that is **understandable** by the computer.



## Programming languages

The goal of a **programming language** is to write instructions in a way that is **understandable** by the computer.

### A programming language can be:

- **Compiled:** machine code is generated from source code;
- **Interpreted:** code can be read directly by the computer without the need of transforming it.

# What is Python ?



## Python

Python is a **high-level, general-purpose programming language**, which emphasizes code readability.

Python is **interpreted** and needs an **interpreter** to be executed on the target system.

Python is widely used across the academic and industrial community:

- Leader in Data Science/Deep Learning;
- Web applications;
- Client/Server applications
- And many more applications ...!

# Why Python?

---

This seminar focuses on Python (instead of R, which would be the closest concurrent) because:

- Very easy to learn and understand: **Readable** syntax, similar to English
- Can be scripted or properly packaged to be production ready;
- Many available libraries for Data Science;
- Standard in NLP and computational linguistics
- Dedicated libraries: NLTK, spaCy, etc.
- Large academic community
- Free and open source

# Example

---

## First Python program

```
1  # This is a comment
2  text = "Computational Philology"
3  # Display a message
4  print("Hello!")
5  print("Number of characters:", len(
    text))
6  # A simple loop
7  words = text.split()
8  for word in words:
9      print("- ", word)
```

## Output

```
Hello!
Number of characters:  27
- Computational
- Philology
```

## Key takeaways:

- `#` comment (ignored)
- `print()` display
- `len()` length
- `for` loop

# Installing Python

---



# We won't install Python in this seminar



## Using Jupyter notebooks

To facilitate access to this seminar, we will not install Python on your machines, but will instead use an interactive programming platform based on Jupyter notebooks, which allow you to combine code, text, visualizations, and comments.

# We won't install Python in this seminar



## Using Jupyter notebooks

To facilitate access to this seminar, we will not install Python on your machines, but will instead use an interactive programming platform based on Jupyter notebooks, which allow you to combine code, text, visualizations, and comments.

## If you want to install Python,

You need to:

- Visit the official Python website: <https://www.python.org>
- Manage virtual environments.
- Use `pip` to install packages.

# Working with Jupyter

---

## Jupyter notebooks

Jupyter Notebooks are **interactive, web-based environments** for creating and sharing documents containing live code, equations, visualizations, and narrative text.

Jupyter notebooks are widely used for **data analysis, scientific computing and machine learning**. It also provides easy sandboxing environment to learn Python!

- The Jupyter Notebook interface consists of cells.
- Cells can contain code or text (Markdown).
- You can execute code cells individually.

- To execute a code cell, select it and press `Shift + Enter`.
- Output will be displayed below the cell.
- Variables and outputs persist across cells within the same notebook.
- You can also interrupt or restart the kernel, which clears all variables and resets the notebook's state.

- Markdown cells allow you to include formatted text, headings, lists, links, images, and more.
- They provide rich narrative capabilities to document your code and analysis.

- Jupyter Notebooks can be saved in the `.ipynb` file format.
- Notebooks can be easily shared with colleagues or published online via platforms like GitHub, Jupyter Notebook Viewer, or Jupyter Hub.
- Notebooks can also be exported to various formats, including HTML, PDF, LaTeX, and slides.



# Practicalities for labs

---

## Connect to the JupyterHub of the hu-berlin

- Go to <https://jupyterhub.cms.hu-berlin.de/>
- Connect with your HU ID and Password
- Select 'Datascience Jupyterlab mit python 3.13' (by default) and click on **START**

## Connect to the Lab's GitHub

- In the menu, click on 'Git' > 'Clone a repository'.
- Copy and paste the following URL:  
`https://github.com/Humboldt-TheoLab/computational\_philology.git`

## Result

The 'computational\_philology' folder should appear in your file section.

# Overview of the two sessions

---

## Session 1 The basics

1. *General introduction* ← here
2. Using a Jupyter notebook
3. Installing packages

## Session 2 Python basics

1. Python Syntax
2. Variables and data types
3. Operators
4. Conditions (if/else)
5. Loops (for, while)
6. Functions
7. Lists and dictionaries
8. Basic OOP

## Goal:

read, modify and write simple Python code to work with textual data.

## Resources

- W3School Python
- *Automate the Boring Stuff*
- <https://pythonhumanities.com/>
- *Intro to Cultural Analytics & Python*: [melaniewalsh.github.io](https://melaniewalsh.github.io)
- [docs.python.org](https://docs.python.org)
- PEP 8 Style Guide for Python Code

## Tip

The best way to learn: **code on your own data.**

Think of a text or corpus you are interested in for the exercises.

# Questions ?

We will now practice using Jupyter and Python !

---